

Utiliser git pour investiguer

Ce TP montre comment on peut utiliser git pour investiguer en cas de problème, notamment à l'aide des commandes `git blame`, `git bisect` et `git show`.

- 1. Prérequis
- 2. Cas de test
 - 2.1. Téléchargement
 - 2.2. Présentation des éléments
 - 2.3. Présentation du bug
- 3. Corriger le bug en mode debug
- 4. Analyser avec git blame
- 5. Trouver la régression avec git bisect
 - 5.1. Dichotomie simple
 - 5.2. Recherche dichotomique et gestion des chemins multiples
 - 5.3. Analyse du commit fautif
- 6. Annexes
 - 6.1. Documentation utile
 - 6.2. Explication de la régression

1. Prérequis

- Disposer de Python3 : fonctionne avec Python 3.12, mais toute version Python3 devrait normalement fonctionner.
- En option, avoir le package `PyYAML` installé, pour pouvoir lire et écrire des fichiers YAML.

2. Cas de test

2.1. Téléchargement

Télécharger le cas de test à l'aide des commandes suivantes :

```
git clone --branch matrix-py --depth 100 https://github.com/alxroyer/edu-sw-dev-env.git matrix/  
git -C matrix/ fetch origin matrix:matrix
```

Explications des commandes git

La première commande réalise 3 choses :

- L'option `--branch` permet de sélectionner la branche `matrix-py` sur laquelle faire *checkout* après le clone.

- L'option `--depth`, couplée avec l'option `--branch` précédente, permet de faire un clone *shallow* (1) de la branche `matrix-py` uniquement.
- Le deuxième argument positionnel, `matrix/`, après l'URL du dépôt, permet d'indiquer le nom du répertoire dans lequel réaliser le clone.

La seconde commande utilise deux options :

- L'option `-C` permet de sélectionner le dépôt fraîchement cloné dans le répertoire `matrix/`.
- La commande `fetch origin matrix:matrix` permet de compléter le clone *shallow*, centré sur la branche `matrix-py`, avec la branche `matrix`, utile également pour ce cas de test.

Notes :

- (1) i.e. un clone ne téléchargeant pas l'intégralité de l'historique du dépôt. Peut s'avérer utile lorsque le dépôt a un historique volumineux, tel que le kernel Linux par exemple.

2.2. Présentation des éléments

Observer l'historique du dépôt téléchargé :

```
cd matrix/
gitk --all &
```

On peut noter l'existence de deux branches dans ce dépôt :

- La branche `matrix`, qui amène progressivement les éléments suivants :
 - Le fichier `Matrix-requirements.md` constituant une spécification du programme *Matrix*, objet de ce cas de test.
 - Des données de test : `A.json`, `B.json`, `C.json` et `D.json`.
 - Le fichier `Matrix-test-plan.md` décrivant un plan de test permettant de couvrir les exigences citées précédemment.
- La branche `matrix-py` réalisant progressivement une implémentation Python du programme *Matrix*.

Observer le document de spécification ainsi que le plan de test.

Lancer le programme Python :

```
python matrix.py
```

Warning PyYAML

Le message de warning suivant "Warning: Please install PyYAML to read and write from YAML files." n'est pas bloquant.

Il rappelle uniquement que le package PyYAML est requis pour pouvoir lire et écrire des fichiers YAML.

En l'absence de PyYAML, le programme reste fonctionnel avec des fichiers JSON.

Tester quelques cas de test décrits dans le plan de test.

2.3. Présentation du bug

L'intérêt de ce cas de test est de présenter un bug, alors même que le développeur l'assure avec aplomb : "Mais si, pourtant, je suis sûr que ça marche ! Ou en tous cas, je l'ai vu marcher."

Tester le cas de test couvrant l'exigence REQ-SYN-070 : Sur exécution de l'opération $R = A * C + B * D$, on obtient l'erreur suivante : "Error: Matrices must have the same dimensions for addition."

3. Corriger le bug en mode debug

Une des premières options pour corriger un tel bug serait d'investiguer le code Python en utilisant les fonctions de debug d'un IDE notamment (cf. TP - Debugging PyCharm Python.md (TODO)).

Mais comme l'objectif de ce TP est plutôt de montrer comment on peut utiliser git pour aider à la résolution de problèmes, on va considérer que le bug que nous avons n'est pas facile à corriger de façon directe.

4. Analyser avec git blame

Si on suspecte une ligne en particulier dans le code source, on peut utiliser la commande `git blame` :

```
git blame matrix.py
```

Cette commande présente le code source avec pour chaque ligne :

- le hash du dernier commit ayant impacté la ligne,
- le nom de la personne ayant dernièrement modifié la ligne,
- la date de dernière modification de la ligne.

Ces informations peuvent permettre d'obtenir de l'aide sur la raison des dernières modifications d'une ou plusieurs lignes suspectées.

Et si le commit référence des tickets associés (issues, ...), cela fournit encore plus d'informations sur l'objet des dernières modifications.

5. Trouver la régression avec git bisect

Le développeur a la certitude que le programme a fonctionné, au moins à un moment. Si on en croit ses mots, le problème observé serait donc une régression.

Lorsqu'on n'arrive pas à trouver facilement les lignes fautives d'une régression, soit en débbuguant, soit avec `git blame`, alors `git bisect` peut probablement apporter des réponses.



`git bisect` apporte un accompagnement pour naviguer dans l'historique des commits, en posant des marqueurs *good* et *bad* progressivement, pour arriver à identifier finalement le commit responsable de la régression.

Le terme *bisect* fait référence au principe de *dichotomie*, ou l'art de *couper* un lot de travail en *deux* pour faciliter la tâche.

Principe de fonctionnement général :

1. Deux marqueurs *good* et *bad* initiaux doivent être déterminés : avant et après la régression.
 - Lorsque le problème a été relevé en production, on pourra typiquement utiliser deux tags de versions.
 - En intégration continue, on pourra se baser sur des résultats de non-régressions à dates.
2. `git bisect` nous positionne sur un commit **au milieu** des deux marqueurs *good* et *bad* encadrant la liste des commits candidats.
3. `git bisect` attend qu'on lui indique si le commit courant doit être considéré comme *good* ou *bad*.
4. Si les commits des marqueurs *good* et *bad* se suivent directement, alors le commit *bad* est le commit responsable de la régression. Fin de la recherche dichotomique en ce cas.
5. Sinon, retour au point 2.

Lorsque plusieurs chemins sont possibles dans l'arbre de versions (cas des merges), le raisonnement nécessite en plus d'explorer les différentes branches, ce que `git bisect` gère très bien automatiquement pour nous.

Comme on va se balader dans l'historique git, et qu'on n'aura pas toujours les données de test *A.json*, *B.json*, ... à disposition, commençons par copier ces données dans un répertoire temporaire :

```
mkdir data
cp *.json data/
```

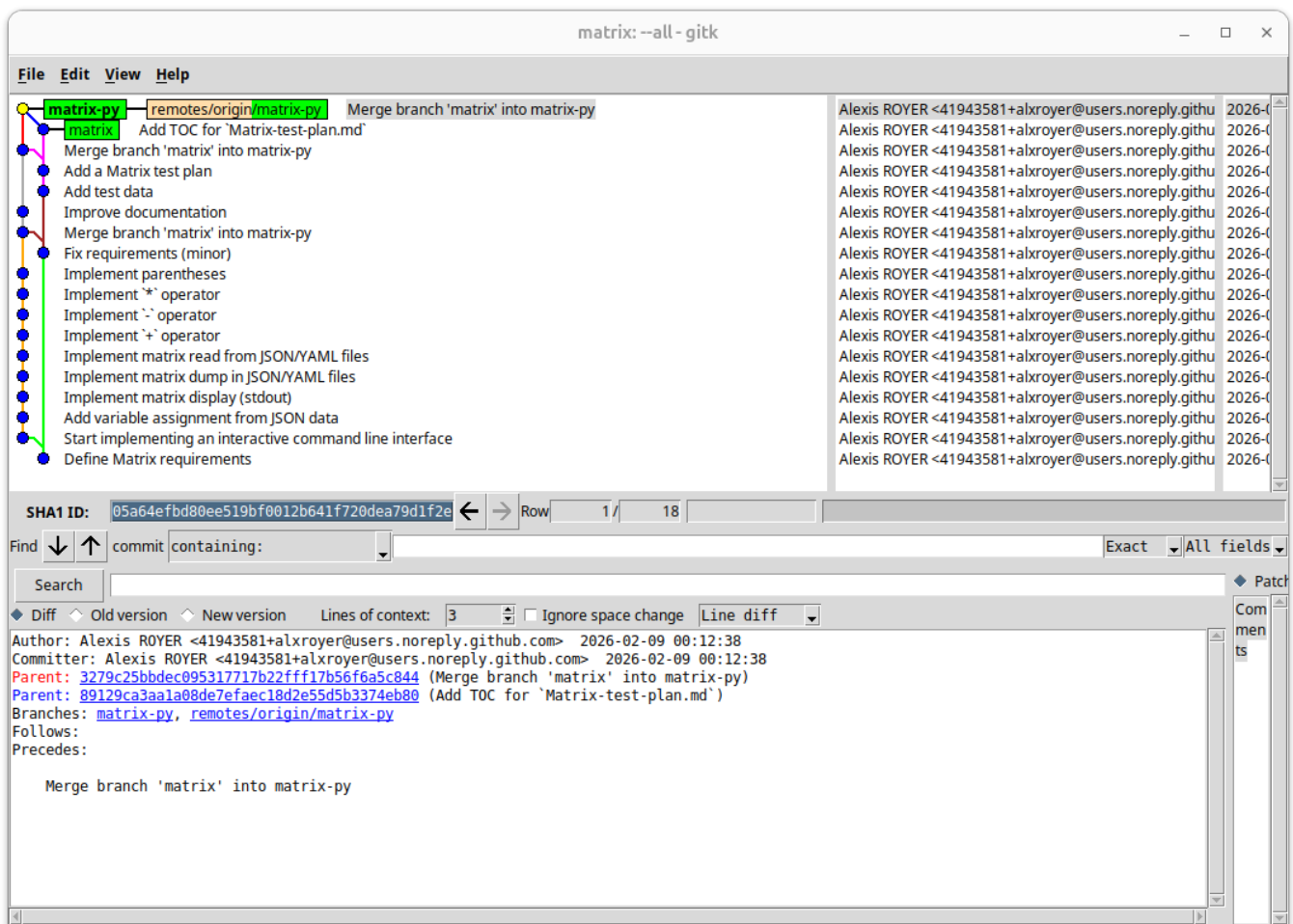
et adaptons les commandes permettant de mettre en évidence le problème identifié :

```
A = read("data/A.json")
B = read("data/B.json")
C = read("data/C.json")
D = read("data/D.json")
R = A * C + B * D
print(R)
exit
```

5.1. Dichotomie simple

Lancer `gitk`, de sorte à pouvoir observer le comportement de git au cours des opérations suivantes :

```
gitk --all &
```



Au démarrage, on repère le point jaune qui nous indique que nous voyons actuellement le dernier commit de la branche `matrix-py`.

A partir de la racine du dépôt, démarrer la recherche dichotomique :

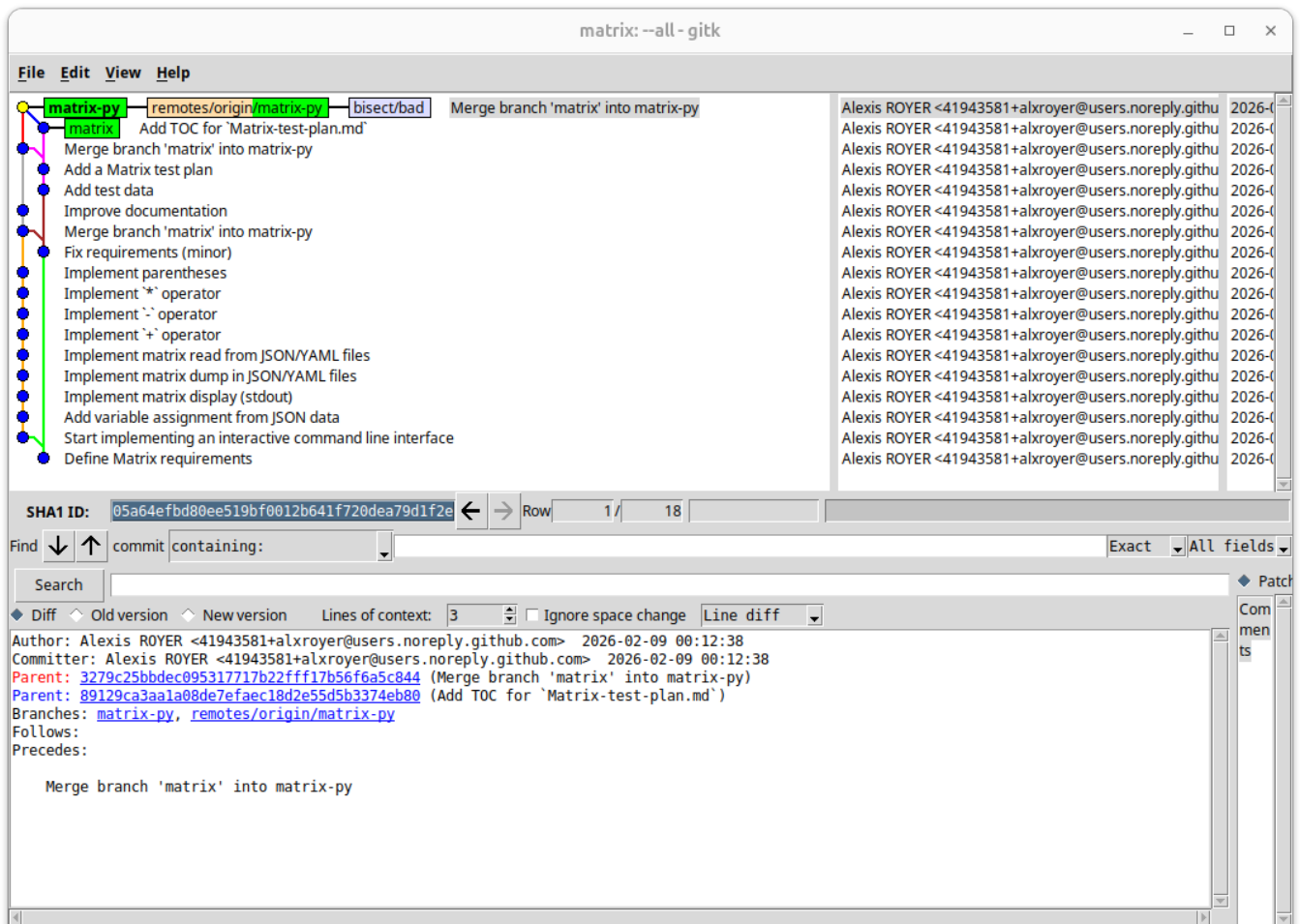
```
git bisect start
```

La commande nous renvoie le message "status: waiting for both good and bad commits" dans la console. Il nous faut effectivement donner deux premiers repères *good* et *bad* pour que `git bisect` puisse démarrer sa recherche dichotomique.

Nous avons déjà identifié précédemment que le bug était présent en l'état du dépôt, à savoir sur le dernier commit de la branche `matrix-py`. Indiquons-le à l'aide de la commande suivante :

```
git bisect bad
```

Rafraîchir la fenêtre gitk à l'aide du raccourci SHIFT+F5 :



Le point jaune est toujours situé sur le dernier commit de la branche `matrix-py`, mais on voit apparaître un marqueur `bisect/bad` en bleu.

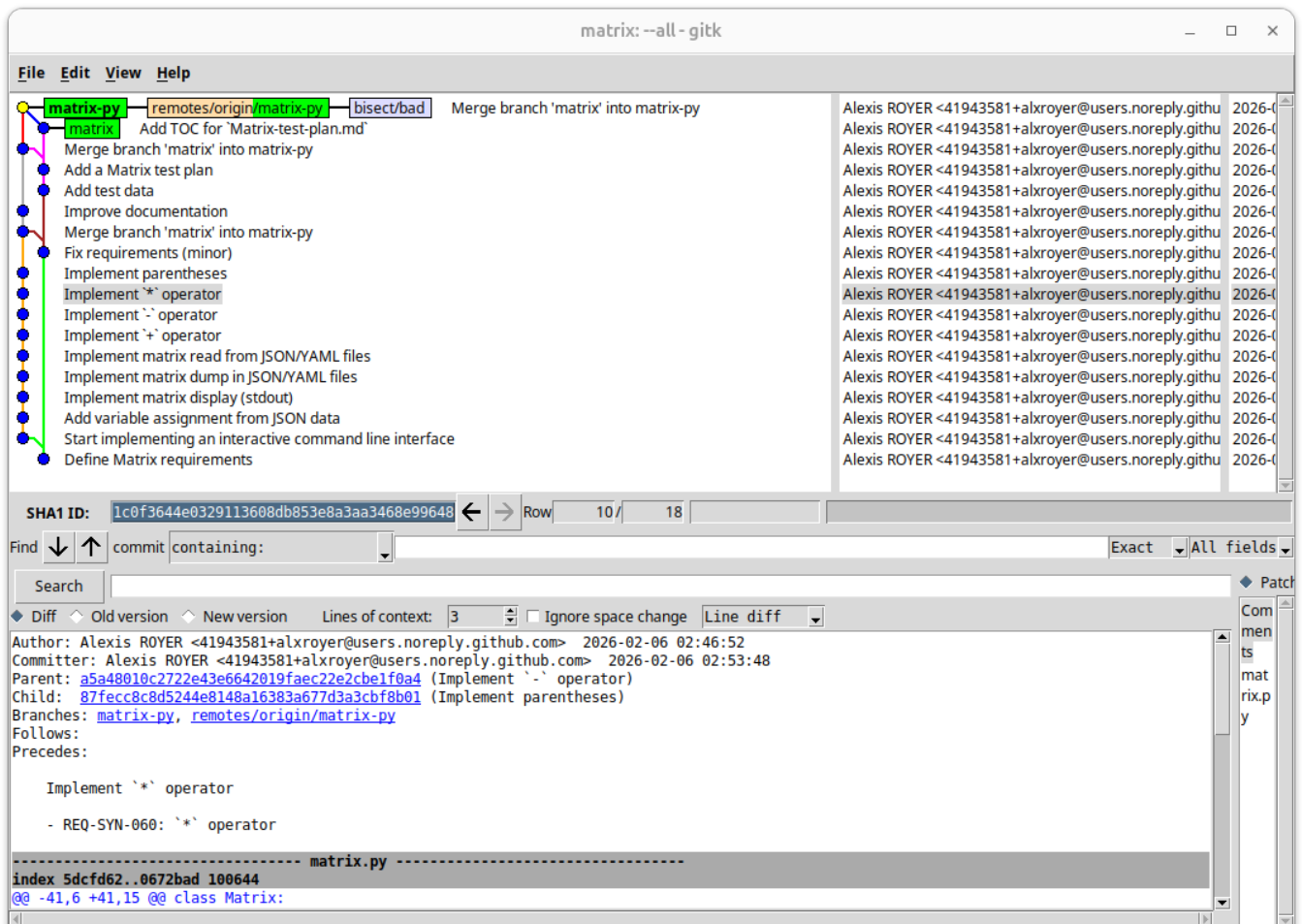
Noter que la commande `git bisect bad` nous a renvoyé le message "status: waiting for good commit(s), bad commit known" dans la console. En effet, nous avons positionné un marqueur `bad`, mais pas encore de marqueur `good`.

Observer l'arbre de versions pour imaginer sur quel commit nous avons une chance de ne pas avoir le bug. Les fonctionnalités ont été développées les unes après les autres :

- "Implement + operator"
- "Implement - operator"
- "Implement * operator"
- "Implement parentheses"

Pour notre cas de test, nous avons besoin des opérateurs `+` et `*`, mais pas nécessairement des parenthèses. Donc si le développeur a le souvenir d'avoir vu fonctionner le programme correctement pour l'opération visée, on peut supposer que dès lors que l'opérateur `*` a été implémenté, le code était fonctionnel.

Dans gitk, cliquer sur le commit identifié, et copier/coller le hash du commit, en l'occurrence `1c0f364` :



Faire un checkout sur le commit identifié :

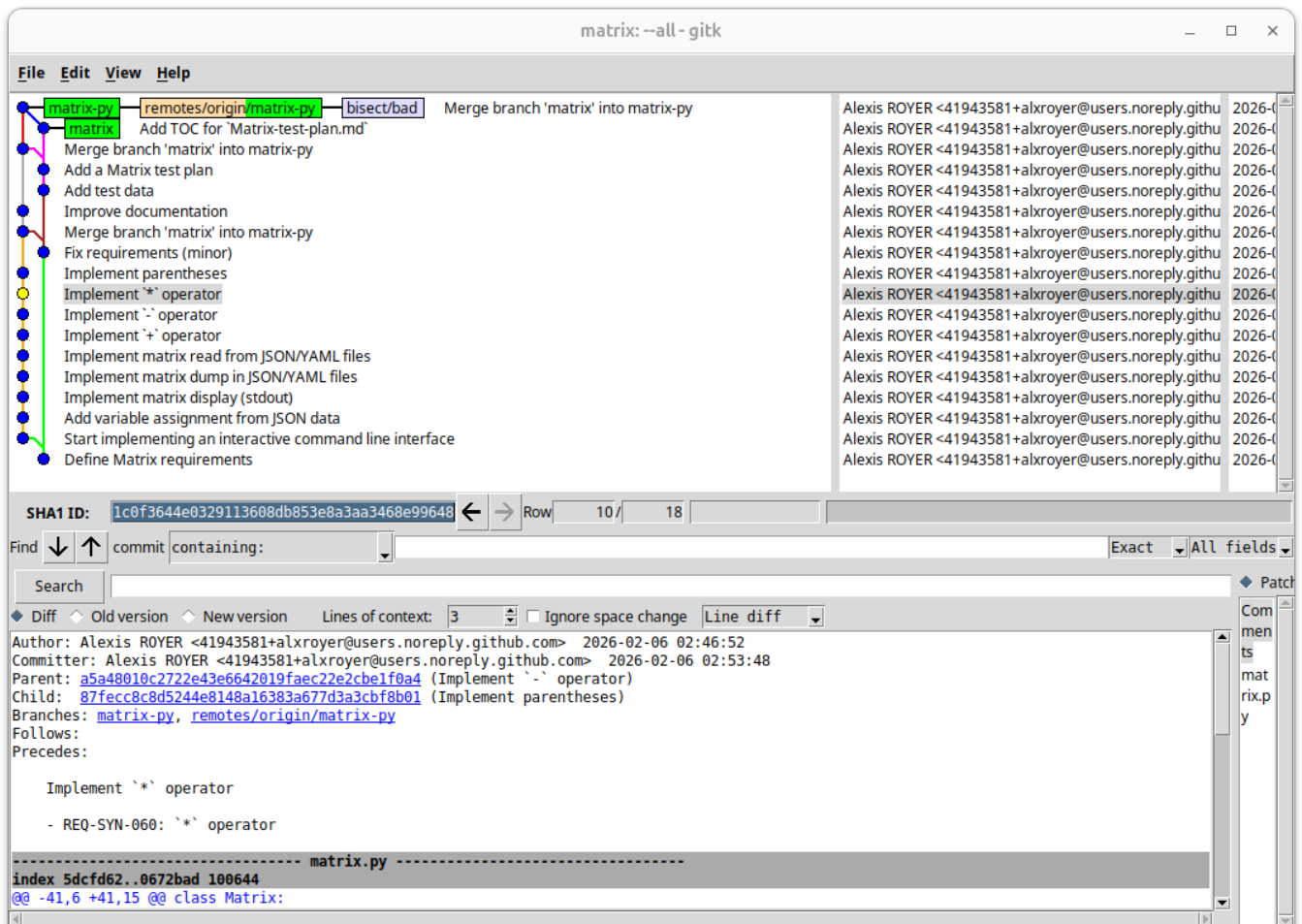
```
git checkout 1c0f364
```

i Warning "detached HEAD"

En faisant le checkout d'un hash de commit (ou d'un tag), git affiche un warning "You are in 'detached HEAD' state".

Ce warning sert à rappeler que le fait de ne pas être en checkout sur une branche rendra plus compliqué la gestion des éventuelles modifications qu'on ferait dans le dépôt.

Ce n'est pas grave dans notre cas, car nous allons seulement exécuter le script `matrix.py`.



Le point jaune est bien passé sur le commit souhaité.

Exécuter le test :

```
python matrix.py
```

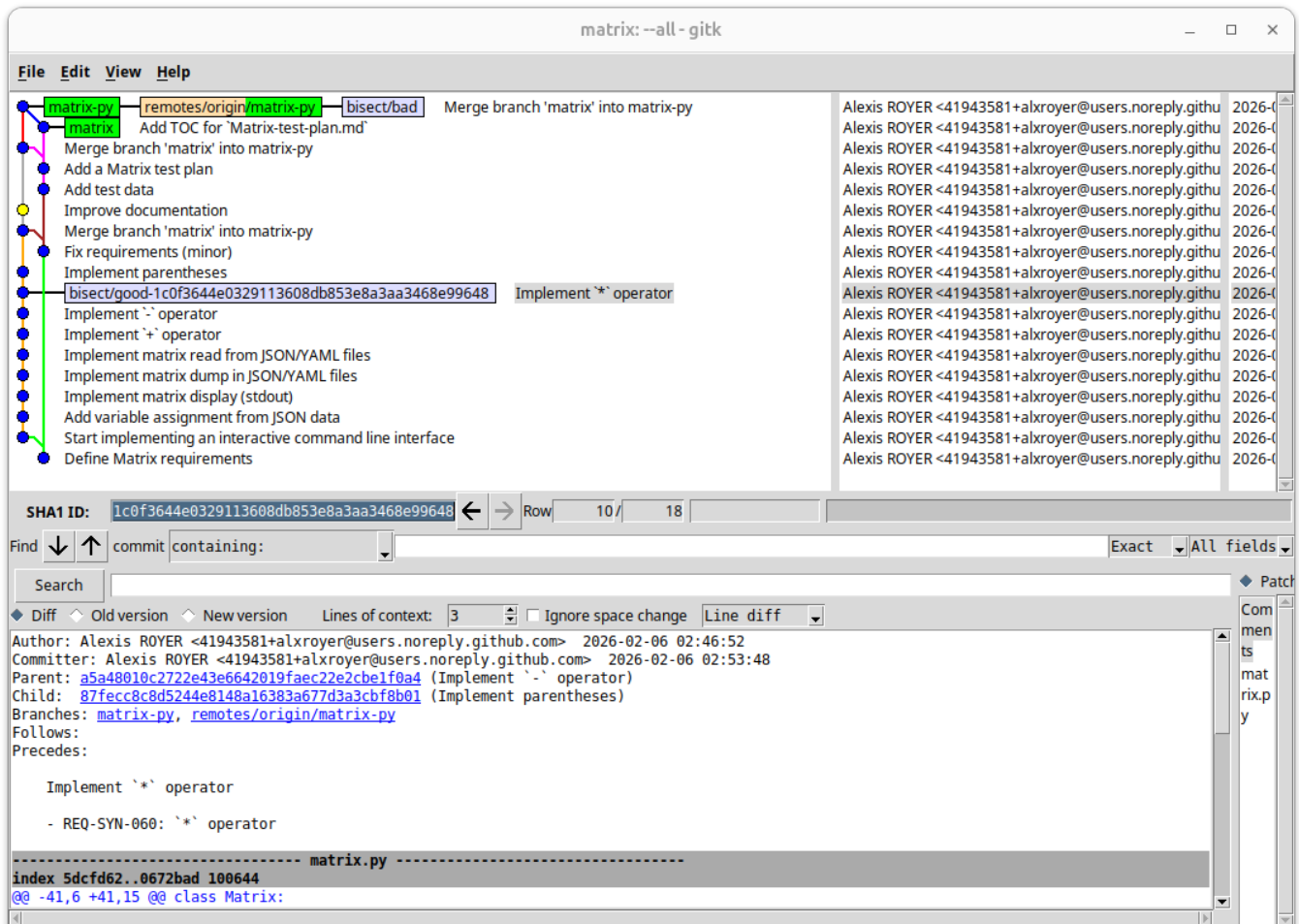
Copier / coller le jeu de commandes identifié précédemment pour essayer de reproduire le problème :

```
matrix> A = read("data/A.json")
Assigned A = [[1, 2], [3, 4]]
matrix> B = read("data/B.json")
Assigned B = [[5, 6], [7, 8]]
matrix> C = read("data/C.json")
Assigned C = [[1], [0]]
matrix> D = read("data/D.json")
Assigned D = [[0], [1]]
matrix> R = A * C + B * D
Assigned R = [[7], [11]]
matrix> print(R)
7
11
matrix> exit
```


Il semble effectivement que, pour le commit identifié, le programme fonctionnait comme attendu. Indiquons-le à `git bisect` :

```
git bisect good
```

Rafraîchir la fenêtre gitk à l'aide du raccourci SHIFT+F5 :



On a un marqueur *good* en bleu qui s'est ajouté sur le commit `1c0f364`. Mais on remarque aussi que le point jaune s'est déplacé sur un autre commit, à peu près au milieu entre les deux marqueurs *good* et *bad*.

En effet, la commande `git bisect good` nous avait renvoyé les messages suivants dans la console :

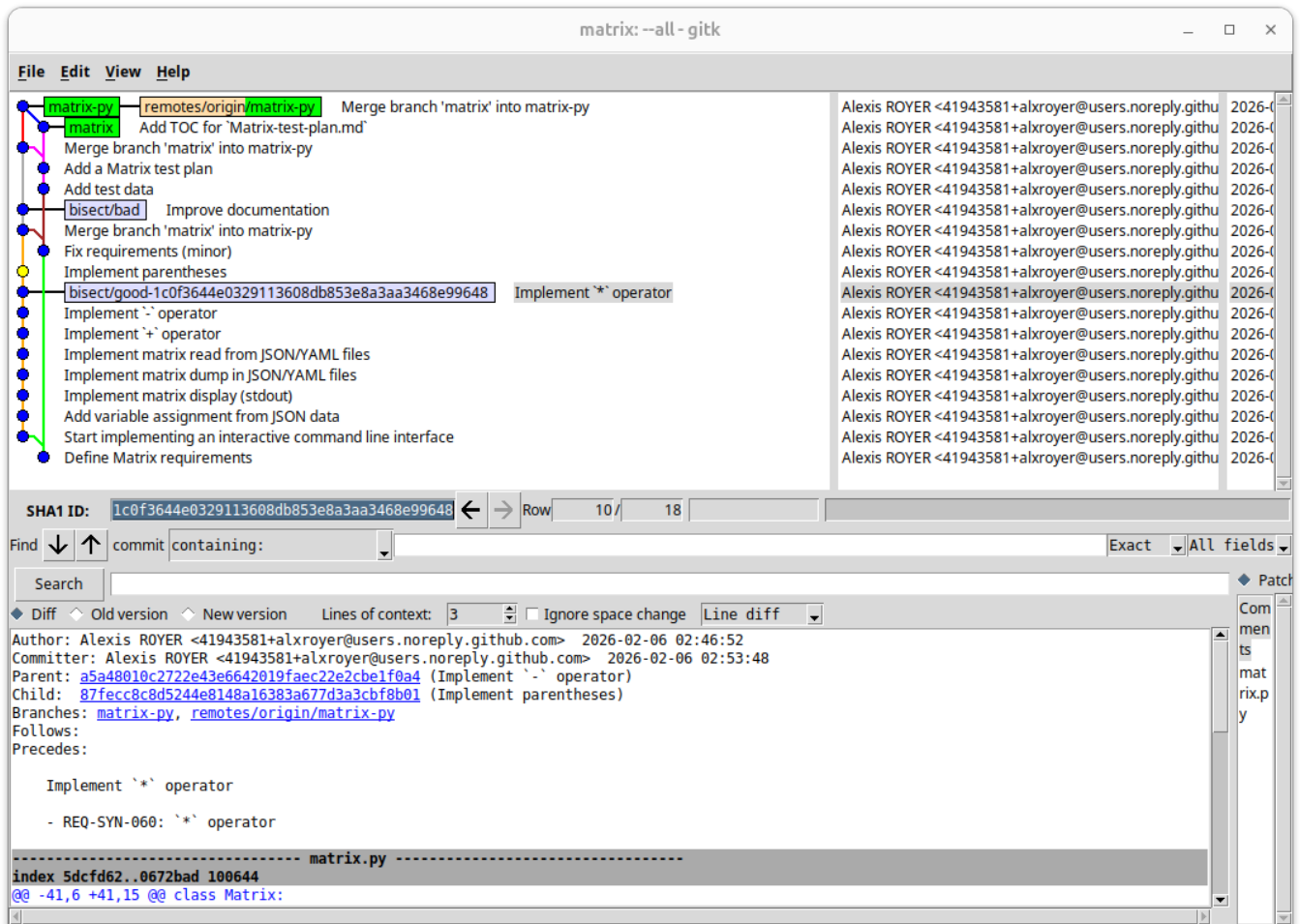
- "Bisecting": Une fois un marqueur *good* et un marqueur *bad* positionnés, `git bisect` peut démarrer son travail de recherche dichotomique.
- "4 revisions left to test after this (roughly 2 steps)": `git bisect` nous fait une estimation du reste à parcourir pour la recherche dichotomique en cours.
- "[817413d927b981e458ecb9d73ada051529b08539] Improve documentation": Reference et commentaire du commit sur lequel `git bisect` a automatiquement fait un checkout, entre les marqueurs *good* et *bad*.

Dans cette situation, `git bisect` attend que nous refassions le test pour lui indiquer si :

- le bug est présent, auquel cas saisir la commande `git bisect bad`,
- ou le bug est absent, auquel cas saisir la commande `git bisect good`.

On refait le test : on retrouve le bug !

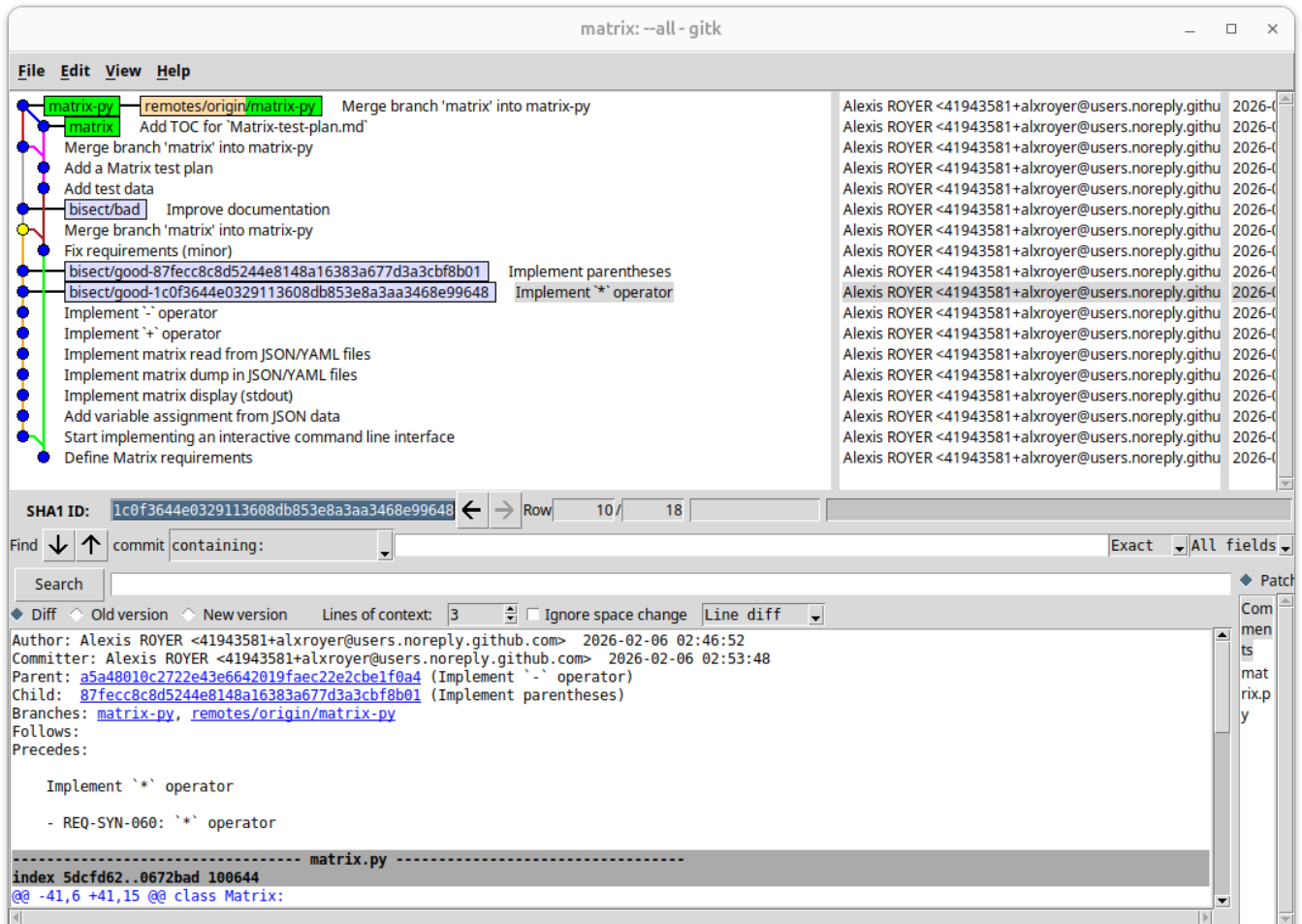
```
git bisect bad
```



Le marqueur *bad* est descendu sur le dernier commit testé. `git bisect` nous a repositionné sur un autre commit entre les marqueurs *good* et *bad*.

On refait le test : pas de bug.

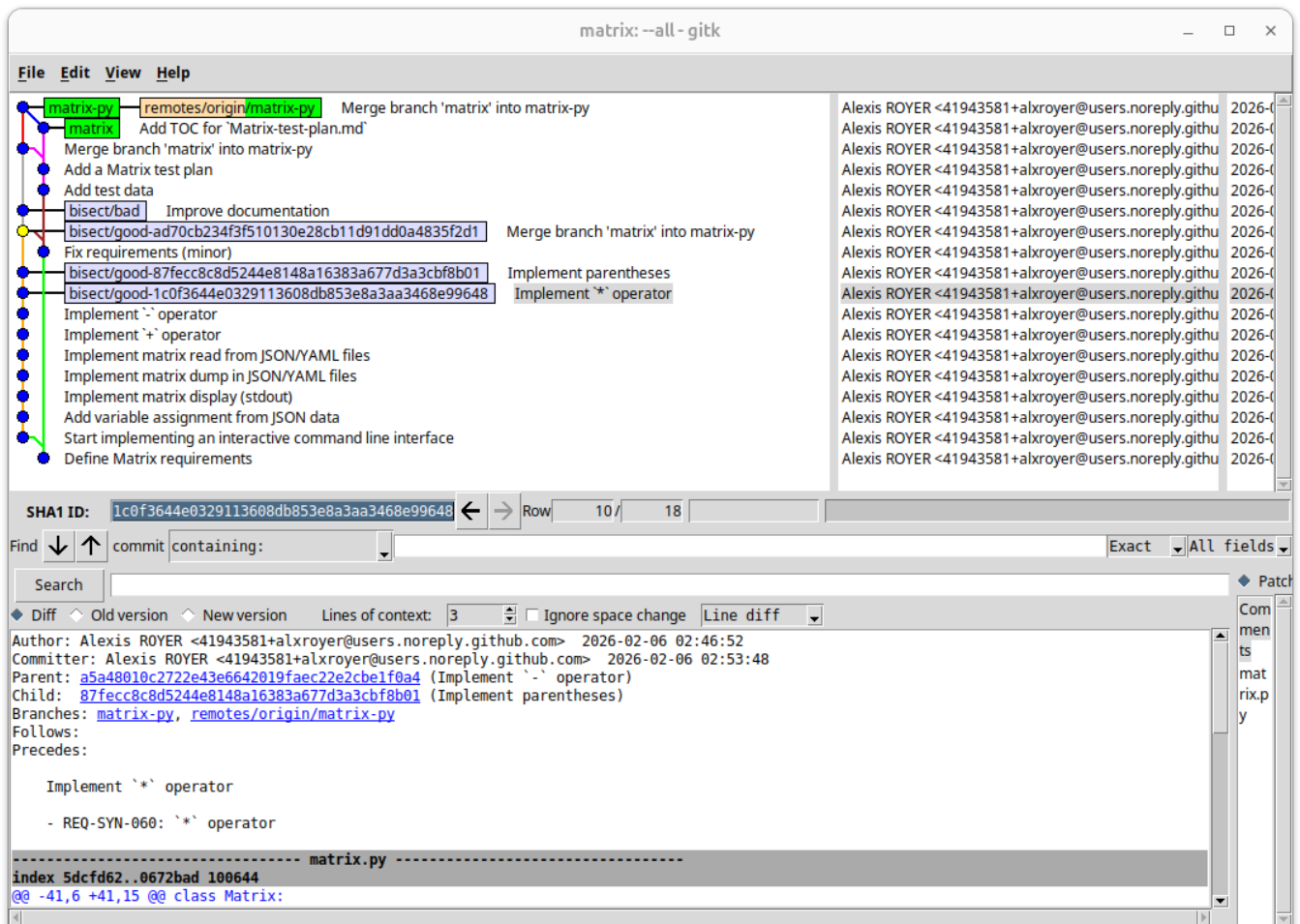
```
git bisect good
```



Un nouveau marqueur *good* a été positionné. `git bisect` nous a repositionné sur un autre commit entre les marqueurs *good* et *bad*.

On refait le test : pas de bug.

```
git bisect good
```



Cette fois-ci, la commande nous affiche les informations d'un commit : le commit fautif, **817413d** ("Improve documentation").

Une fois le commit identifié, on peut stopper la recherche dichotomique :

```
git bisect reset
```

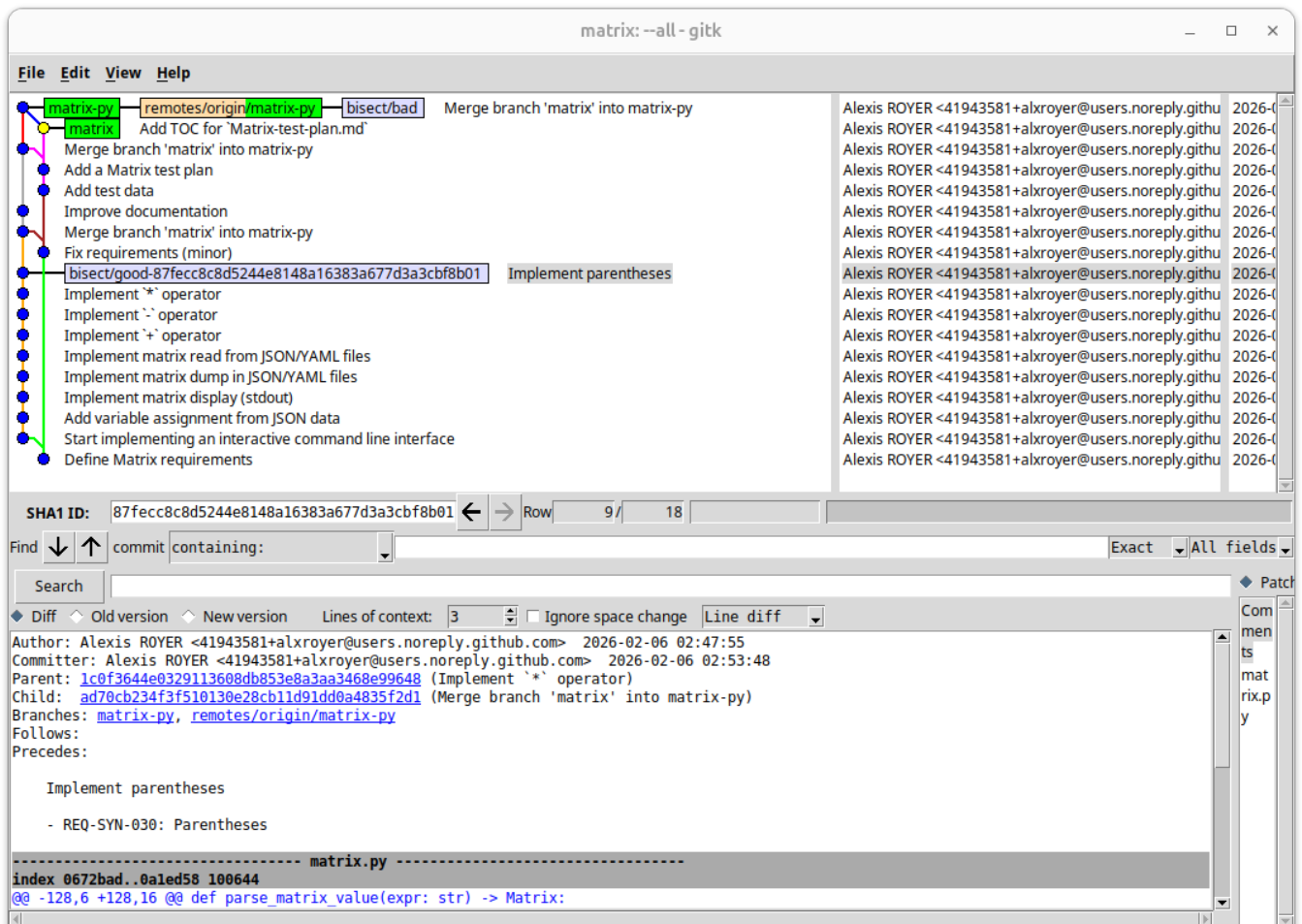
5.2. Recherche dichotomique et gestion des chemins multiples

La recherche dichotomique précédente s'était résolue assez facilement, de par les noeuds parcourus de façon linéaire, sur la branche **matrix-py** uniquement.

Relancer la recherche dichotomique, en prenant le commit **87fecc8** ("Implement parentheses") comme premier commit *good* :

```
git bisect start
git bisect bad
git checkout 87fecc8
git bisect good
```

Observer la situation dans gitk :



On constate que `git bisect` nous a positionné sur un commit de la branche `matrix` cette fois-ci. Et oui, pourquoi pas ? Rien n'interdit dans l'absolu que le bug puisse venir de la branche `matrix`.

Après réflexion non, car le programme `matrix.py` n'existe pas dans la branche `matrix`, c'est pourquoi il faut répondre `good` dans cette situation.

```
git bisect good
```

Poursuivre la recherche dichotomique : `git bisect` finit par pointer le même commit fautif que précédemment, le `817413d` ("Improve documentation").

5.3. Analyse du commit fautif

Observer le commit log du commit identifié : "Improve documentation". Cela nous donne le contexte de l'intention du commit. En l'occurrence, et en toute logique, le fait d'améliorer la documentation ne devrait pas changer le fonctionnel.

Pour rappel, le commit log peut également donner des références de tickets associés (cf. [TP - Git strategy.md](#)). Ces tickets peuvent être une source d'information très utile dans ce genre de situation. Ici toutefois, pas de référence de ticket.

Observer le détail des modifications associées à ce commit, soit dans gitk, soit avec une commande `git show` comme suit :

```
git show 817413d
```

```
commit 817413d927b981e458ecb9d73ada051529b08539
Author: Alexis ROYER <41943581+alxroyer@users.noreply.github.com>
Date:   Fri Feb 6 03:02:45 2026 +0100

    Improve documentation

diff --git a/matrix.py b/matrix.py
index 0a1ed58..26590b6 100644
--- a/matrix.py
+++ b/matrix.py
@@ -19,12 +19,16 @@ class Matrix:
     def __repr__(self) -> str:
         return repr(self.data)

-    # REQ-SYN-100: Stdout matrix output
+    def __str__(self) -> str:
+        """
+        REQ-SYN-100: Stdout matrix output
+        """
+        return '\n'.join([' '.join(map(lambda x: f"{x:>5}", row)) for row
+in self.data])

-    # REQ-SYN-040: `+` operator
+    def __add__(self, other: 'Matrix') -> 'Matrix':
+        """
+        REQ-SYN-040: `+` operator
+        """
+        if len(self.data) != len(other.data) or len(self.data[0]) !=
len(other.data[0]):
+            raise ValueError("Matrices must have the same dimensions for
addition.")
+        return Matrix([
@@ -32,8 +36,10 @@ class Matrix:
         for i in range(len(self.data))
         ])

-    # REQ-SYN-050: `-` operator
+    def __sub__(self, other: 'Matrix') -> 'Matrix':
+        """
+        REQ-SYN-050: `-` operator
+        """
+        if len(self.data) != len(other.data) or len(self.data[0]) !=
len(other.data[0]):
+            raise ValueError("Matrices must have the same dimensions for
```



```

subtraction.")
        return Matrix([
@@ -41,8 +47,10 @@ class Matrix:
        for i in range(len(self.data))
        ])

-   # REQ-SYN-060: `*` operator
    def __mul__(self, other: 'Matrix') -> 'Matrix':
+       """
+       REQ-SYN-060: `*` operator
+       """
        if len(self.data[0]) != len(other.data):
            raise ValueError("Number of columns in the first matrix must
equal number of rows in the second matrix for multiplication.")
        return Matrix([[
@@ -55,9 +63,11 @@ class Matrix:
        data = json.loads(json_data)
        return cls(data)

-   # REQ-SYN-022: File matrix input
    @classmethod
    def from_file(cls, file_path: str) -> 'Matrix':
+       """
+       REQ-SYN-022: File matrix input
+       """
        with open(file_path, 'r') as file:
            if file_path.endswith('.json'):
                data = json.load(file)
@@ -67,8 +77,10 @@ class Matrix:
            raise ValueError("Unsupported file format.")
        return cls(data)

-   # REQ-SYN-110: File matrix output
    def to_file(self, file_path: str):
+       """
+       REQ-SYN-110: File matrix output
+       """
        with open(file_path, 'w') as file:
            if file_path.endswith('.json'):
                json.dump(self.data, file)
@@ -79,6 +91,12 @@ class Matrix:

    def parse_line(line: str) -> str:
+       """
+       Parses a line from the interactive command line interface.
+       :param line: Line to parse.
+       :return: Result message.
+       """
        _match: re.Match[str]

        # REQ-SYN-010: Variable assignment

```



```

@@ -108,14 +126,22 @@ def parse_line(line: str) -> str:
    raise SyntaxError(f"Invalid syntax {line!r}")

-# REQ-SYN-020: Matrix value
def parse_matrix_value(expr: str) -> Matrix:
+    """
+    REQ-SYN-020: Matrix value
+
+    :param expr: Expression to parse.
+    :return: Matrix computed from ``expr``.
+    """
    _match: re.Match[str]
    _tmp_var_name: str

-    # REQ-SYN-023: Variable value
-    if expr in variables:
-        return variables[expr]
+    # REQ-SYN-021: JSON matrix input
+    if expr.startswith("[") and expr.endswith("]"):
+        try:
+            return Matrix.from_json(expr)
+        except json.JSONDecodeError:
+            raise ValueError(f"Invalid matrix value: {expr!r}")

    # REQ-SYN-022: File matrix input
    _match = re.match(r"^(.*)read\(\s*"([^\"]*)"\s*\)(.*)$", expr)
@@ -128,6 +154,10 @@ def parse_matrix_value(expr: str) -> Matrix:
    finally:
        del variables[_tmp_var_name]

+    # REQ-SYN-023: Variable value
+    if expr in variables:
+        return variables[expr]
+
    # REQ-SYN-030: Parentheses
    _match = re.match(r"^.*\([^\)]+\)(.*)$", expr)
    if _match:
@@ -138,6 +168,13 @@ def parse_matrix_value(expr: str) -> Matrix:
    finally:
        del variables[_tmp_var_name]

+    # REQ-SYN-060: ``*`` operator
+    _match = re.match(r"^.*(\*)(.*)$", expr)
+    if _match:
+        _m1 = parse_matrix_value(_match.group(1).strip())
+        _m2 = parse_matrix_value(_match.group(3).strip())
+        return _m1 * _m2
+
    # REQ-SYN-040: ``+`` operator
    # REQ-SYN-050: ``-`` operator
    _match = re.match(r"^.*([+-])(.*)$", expr)
@@ -149,25 +186,13 @@ def parse_matrix_value(expr: str) -> Matrix:

```

```

        else:
            return _m1 - _m2

- # REQ-SYN-060: `` operator
- _match = re.match(r"^(\.*)(\*)(.*)$", expr)
- if _match:
-     _m1 = parse_matrix_value(_match.group(1).strip())
-     _m2 = parse_matrix_value(_match.group(3).strip())
-     return _m1 * _m2
-
- # REQ-SYN-021: JSON matrix input
- if expr.startswith("[") and expr.endswith("]"):
-     try:
-         return Matrix.from_json(expr)
-     except json.JSONDecodeError:
-         raise ValueError(f"Invalid matrix value: {expr!r}")
-
    raise SyntaxError(f"Invalid syntax {repr!r}")

def main():
- # REQ-UI-010: Interactive command line interface
+ """
+ REQ-UI-010: Interactive command line interface
+ """
    print("Matrix command line. Type 'exit' to quit.")
    while True:
        try:

```

On voit dans ces diffs un certain nombre de modifications :

- Des transformations de commentaires Python en docstrings : ça correspond bien à l'objectif d'amélioration de documentation affiché dans le commit log.
- Des ajouts de docstrings, notamment pour la fonction `parse_line()` : ça correspond toujours à la description du commit log.
- Mais également des restructurations de code dans la fonction `parse_matrix_value()` : et ça, c'est plus suspect !

Avec ces informations, vous saurez probablement mieux expliquer la régression. Et une fois la régression expliquée, la correction n'est plus très loin.

A ce stade, c'en est fini de l'aide que `git` peut nous apporter pour analyser cette régression. C'est donc techniquement la fin de ce TP.

Explication de la régression

Pour véritablement conclure les esprits curieux, voir l'explication de la régression annexes.

6. Annexes

6.1. Documentation utile

Commandes git utiles :

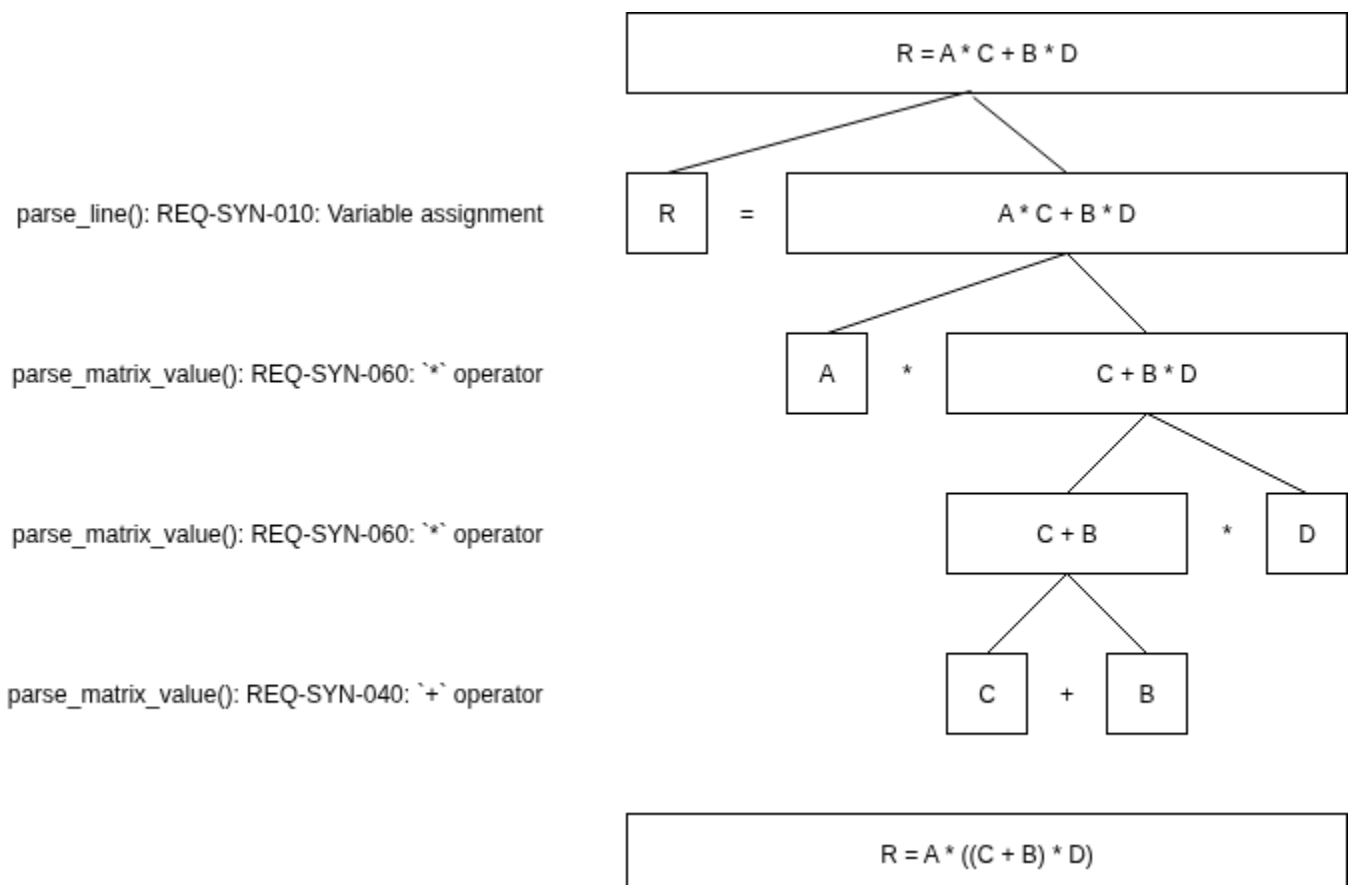
- <https://git-scm.com/docs/git-blame>
- <https://git-scm.com/docs/git-bisect>

6.2. Explication de la régression

Pour ne pas rester sur un goût d'inachevé, expliquons dans cette annexe la régression qui a été introduite dans ce programme Python.

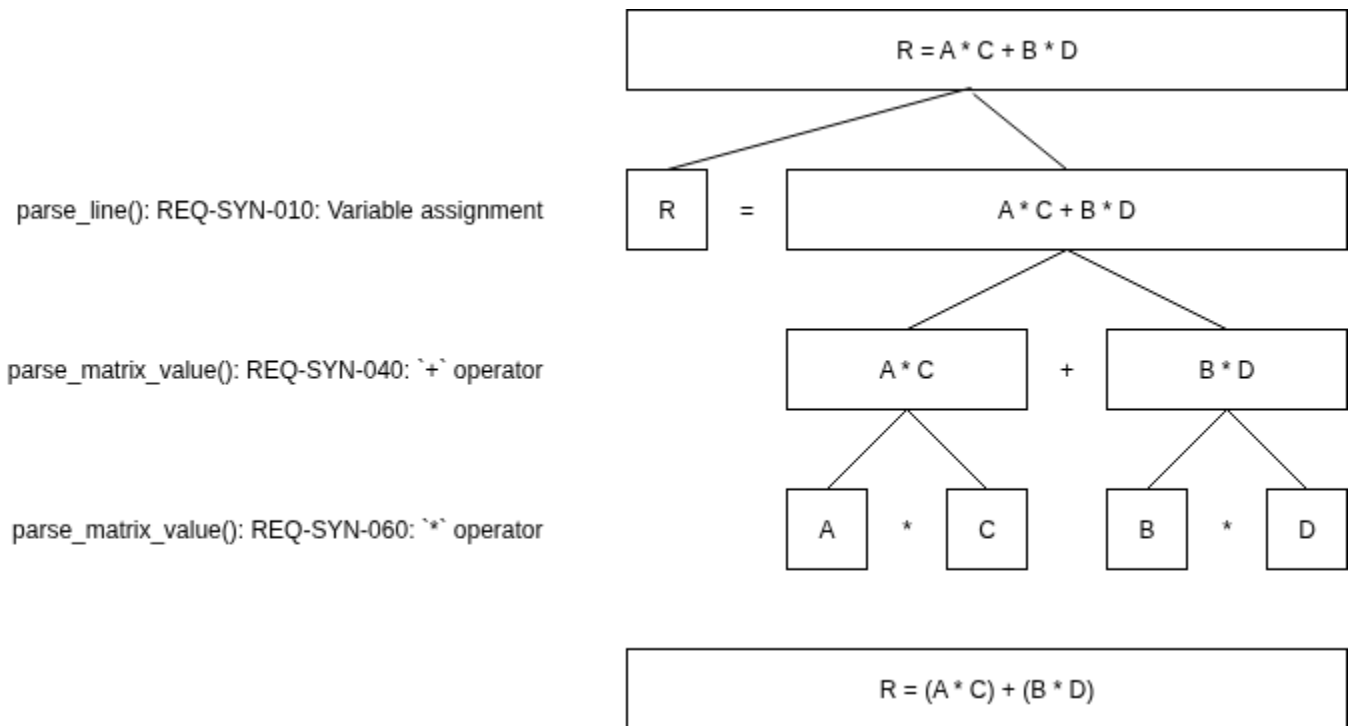
Le commit [817413d](#) ("Improve documentation") étant responsable de la régression, et après analyse des modifications apportées par ce commit, il semble que l'ordre des traitements effectués dans la fonction `parse_matrix_value()` a son importance.

En effet, après le commit [817413d](#), le fonctionnement de l'algorithme sur la formule $R = A * C + B * D$ peut être schématisé comme suit :



De manière contre-intuitive, le fait de traiter les opérateur $*$ *en priorité* dans l'analyse de la ligne de commande relègue les opérations $+$ en bas de l'arbre des traitements, ce qui rend au final ces dernières plus prioritaires dans le calcul.

Pour garder les opérateurs $*$ prioritaires, il convient en réalité de les traiter après les opérations $+$ et $-$ dans l'analyse de commande.



Vous savez désormais comment corriger le programme.